Projeto de TCC · FATEC Adib Moisés Dib

Monitoramento inteligente do motor de uma máquina CNC com TinyML

Explicação simples de o que o projeto faz, por que coleta dados e como o modelo aprende.

1. Em resumo

Instalamos um pequeno computador (ESP32) em uma máquina CNC para medir a eletricidade que o motor consome. Com esses números, um classificador baseado em rede neural (TinyML), executado em inferência embarcada no próprio ESP32, identifica em que estado a máquina está: parada, girando sem cortar, cortando ou sob esforço excessivo.

2. Para que coletar esses dados?

Hoje a máquina não tem nenhum monitoramento automático: só percebemos um problema quando ele já aconteceu. Medindo a corrente do motor o tempo todo, conseguimos saber quanto tempo ele trabalha em cada condição e avisar quando ele sai da faixa segura. Esse é o primeiro passo para a manutenção preditiva e para entender o consumo de energia.

3. O rótulo: o que o manual do motor diz

Para o computador aprender, alguém precisa dizer o que é "normal" e o que não é. Esse "gabarito" se chama **rótulo**. O nosso vem do manual (folha de dados) do fabricante do motor, a WEG (W22 IR3 Premium, 3 cv, 4 polos, código 11397215), que informa em que condições o motor foi projetado para trabalhar:

Dado do fabricante	Valor	Fonte
Corrente nominal (plena carga)	4,78 A em 380 V	Folha de dados, p. 1/5
Corrente a vazio (girando sem carga)	2,66 A em 380 V	Folha de dados, p. 1/5
Fator de serviço (sobrecarga tolerada)	1,25	Folha de dados, p. 1/5
Rotação nominal	1745 rpm	Folha de dados, p. 1/5

A partir da corrente nominal, definimos cinco estados (valores para 380 V):

Estado	Corrente	Significado
PARADA	≈ 0 A	Motor desligado
VAZIO	até 4,0 A (≈ 84% da nominal)	Motor girando, sem cortar a peça
USINANDO	4,0 a 5,3 A	Cortando a peça, esforço normal
SOBRECARGA	acima de 5,3 A (> 110%)	Esforço acima do nominal
CRÍTICO	7,2 A ou mais (≥ 150%)	Risco de dano ao motor

O limite de 4 A entre vazio e usinando foi observado na própria máquina. As demais faixas foram definidas pelos autores a partir da corrente nominal da folha de dados.

4. Como o projeto funciona

1. **Coleta:** o ESP32 lê tensão, corrente, potência, fator de potência, frequência e energia a cada 10 segundos e envia para uma planilha e para a nuvem.
2. **Preparação:** os dados saem da planilha em CSV e cada leitura recebe seu rótulo. Eles são brutos: antes do treino passam por seleção de variáveis e normalização z-score (seção 5).
3. **Treino no computador:** em ambiente controlado, usando Python, o modelo aprende a ligar os números aos rótulos.
4. **Exportação:** os números aprendidos (os "pesos") são copiados para o código do ESP32.
5. **Uso real:** o ESP32 roda o modelo sozinho, em tempo real, sem internet e sem computador, e informa o estado da máquina.

Ferramenta	Para que serve
ESP32 DevKit V1	Microcontrolador que lê os sensores e executa o modelo
PZEM-004T v3.0	Medidor de tensão, corrente, potência e fator de potência
Google Sheets + Apps Script	Guardam o histórico das leituras
MQTT (HiveMQ) e Power BI	Envio em tempo real e painéis de visualização
Python (NumPy, scikit-learn, TensorFlow/Keras)	Preparar os dados e treinar o modelo
Arduino IDE	Escrever e gravar o programa no ESP32

5. Preparação dos dados: variáveis e z-score

Os dados exportados da planilha são **brutos**: têm unidades e escalas muito diferentes (potência em centenas de watts, fator de potência entre 0 e 1, corrente em poucos ampères). Por isso passam por duas etapas antes do treino.

Seleção de variáveis

O modelo rende mais com a **corrente** e com as variáveis que acompanham sua variação, que são as que carregam informação sobre o esforço do motor. Variáveis quase constantes não ajudam e, depois do z-score, teriam o ruído ampliado.

Variável	Comportamento nos dados	Decisão
Corrente	Varia bastante; reflete o esforço do motor	Usada
Potência	Varia junto com a corrente	Usada
Fator de potência	Varia bastante (correlação −0,93 com a corrente)	Usado
Tensão	Varia pouco (correlação −0,22 com a corrente)	Descartada
Frequência	Praticamente constante	Descartada
Energia (kWh)	Contador acumulado: só cresce e mede tempo, não estado	Descartada

Além das três variáveis usadas, calculam-se a variação da corrente entre leituras e a média e o desvio-padrão da corrente na janela móvel de 10 leituras, somando **6 entradas**.

Normalização z-score

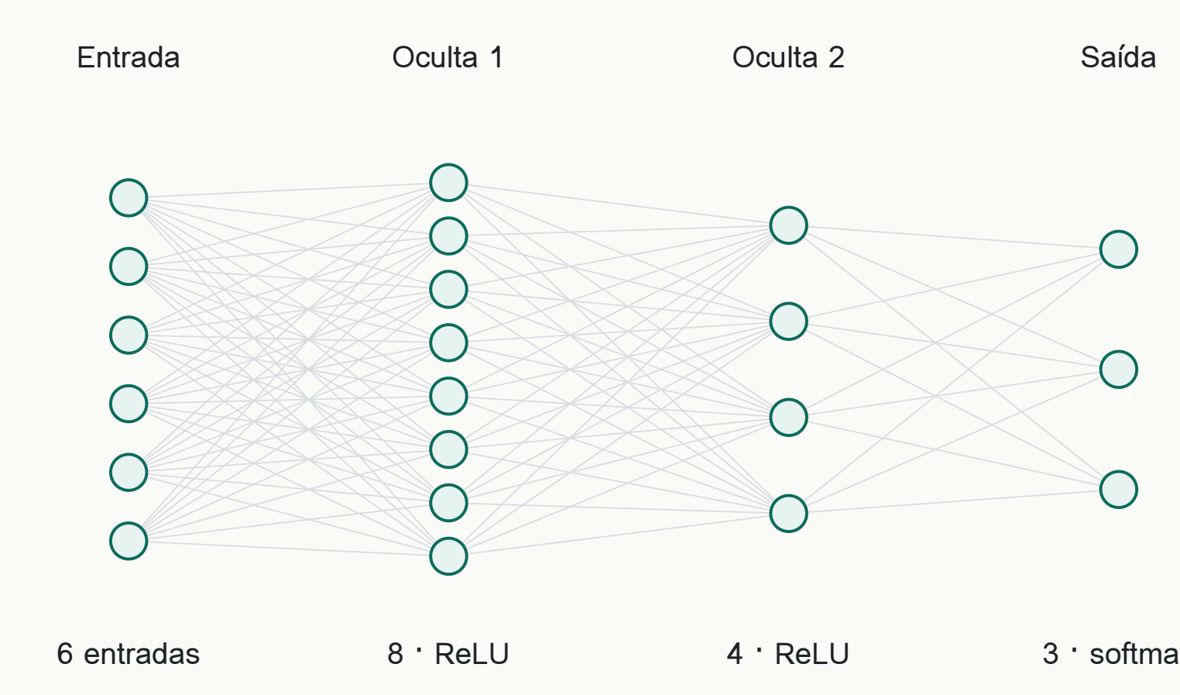
Cada variável é convertida por $\mathbf{z = (x - \mu) / \sigma}$, em que μ é a média e σ o desvio-padrão. Sem isso, a variável de maior escala domina o cálculo do gradiente: seus pesos mudam mais rápido e o modelo passa a ser "puxado" por ela, o que não é o desejado. Com o z-score, todas ficam com média 0 e desvio 1 e contribuem de forma equilibrada.

Exemplo ilustrativo: corrente de 4,5 A ($\mu = 3{,}8$; $\sigma = 0{,}5$) resulta em $z = 1{,}40$; potência de 1.900 W ($\mu = 1.400$; $\sigma = 350$) resulta em $z = 1{,}43$. Valores brutos muito diferentes passam a ter a mesma ordem de grandeza.

- μ e σ são calculados **apenas no conjunto de treino** e gravados no firmware como constantes.
- O ESP32 aplica a mesma conta em cada leitura nova, antes da inferência.
- Os valores de z são limitados a ±8 para evitar que um ponto extremo distorça a rede.

6. O modelo: rede neural MLP

O modelo é um **Perceptron Multicamadas (MLP)** do tipo feedforward, treinado de forma **supervisionada**: cada leitura tem um rótulo e o algoritmo minimiza o erro entre a saída da rede e esse rótulo. A estrutura adotada é **6 → 8 → 4 → 3**, com 107 parâmetros treináveis (pesos e vieses), pequena o bastante para o ESP32.

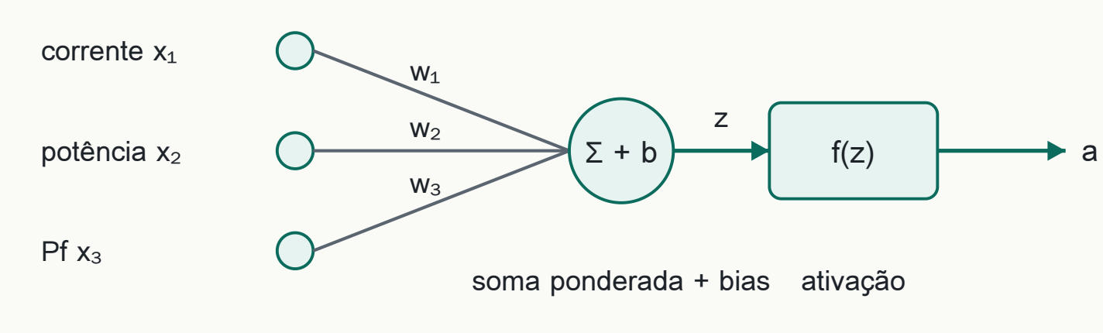


Cada círculo é um neurônio e cada linha é um peso. Entradas (normalizadas por z-score): corrente, potência, fator de potência, variação da corrente e média e desvio-padrão da corrente na janela móvel de 10 leituras. Saídas: PARADA, VAZIO e USINANDO; SOBRECARGA e CRÍTICO ficam por limiar fixo no firmware.

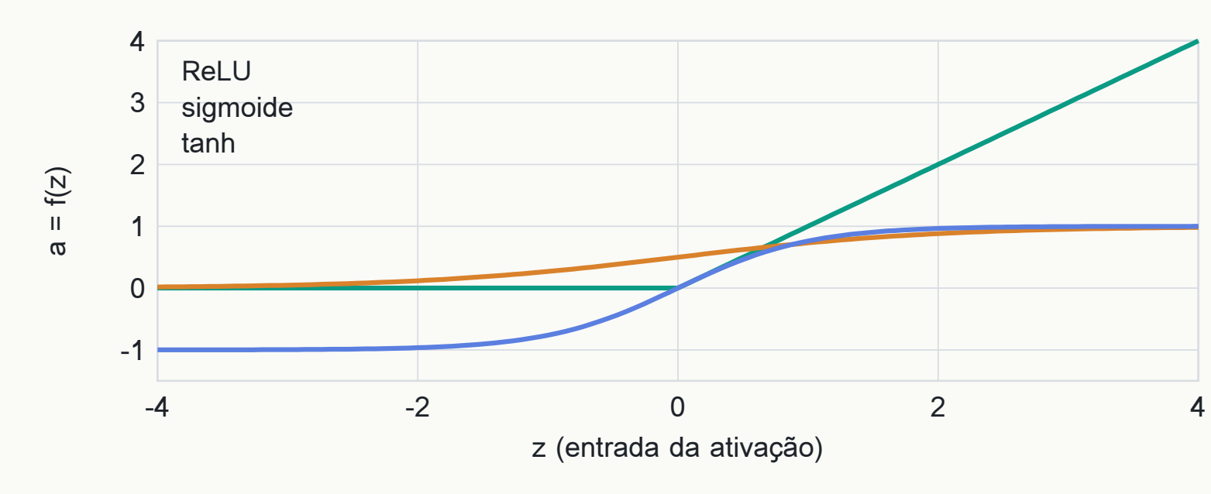
Cada neurônio calcula $\mathbf{z = \Sigma(w_i \cdot x_i) + b}$ e aplica uma função de ativação, $\mathbf{a = f(z)}$. Uma camada inteira é $\mathbf{a = f(W \cdot x + b)}$, uma multiplicação matricial. É só isso que o ESP32 executa na inferência (propagação direta).

Função de ativação

Cada neurônio faz duas operações. Primeiro a **soma ponderada**, $z = w_1x_1 + w_2x_2 + w_3x_3 + b$, que é a equação de uma reta (w é a inclinação e b o deslocamento). Depois aplica a **ativação**, $a = f(z)$, que transforma essa reta em uma curva.



Sem a ativação, várias camadas somadas continuariam sendo uma única reta, e a rede não conseguiria separar estados como VAZIO e USINANDO. A curva permite "dobrar" a fronteira entre eles.



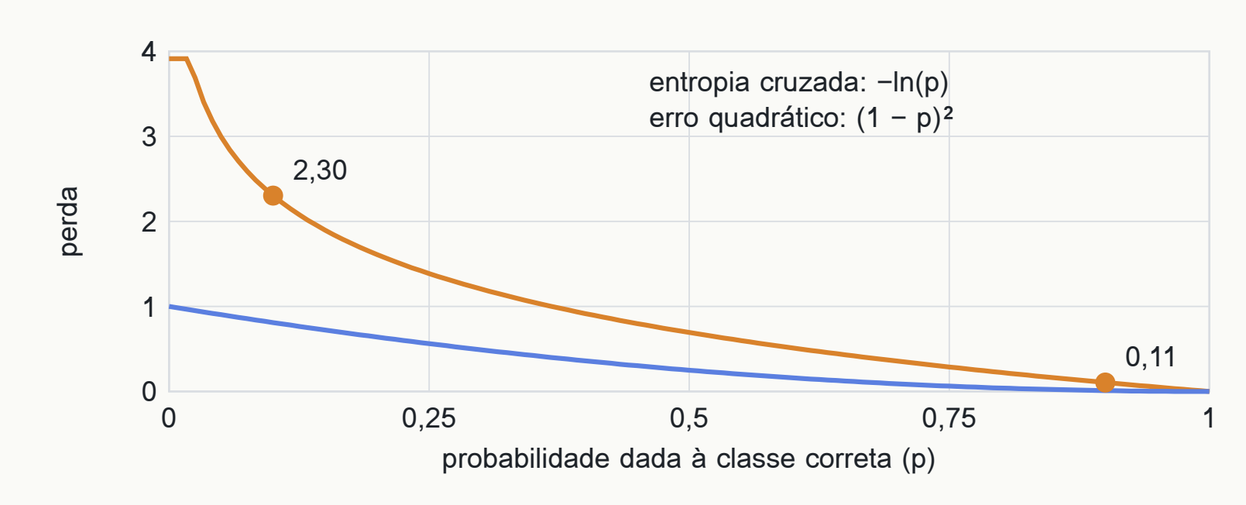
A sigmoide (0 a 1) e a tanh (−1 a 1) achatam nas pontas: ali a curva é quase horizontal, a derivada é próxima de zero e o aprendizado emperra. A **ReLU**, $f(z) = \max(0, z)$, vale zero para valores negativos e é uma reta para os positivos; é mais barata (uma comparação) e não achata. Foi a escolhida para as camadas ocultas.

Na saída usa-se a **softmax**, que converte as pontuações em probabilidades que somam 100%. Exemplo: pontuações USINANDO 2,0, VAZIO 1,0 e PARADA 0,1 viram 66%, 24% e 10%. O estado informado é o de maior probabilidade.

Função de perda

Depois de cada previsão, a rede precisa de um número que diga o quão errada ela está: a **função de perda**. Ela responde (1) houve erro? (2) de que tamanho? e serve de guia para (3) o aprendizado. Em problemas de valor contínuo usam-se o MAE (erro absoluto, peso proporcional ao erro), o MSE (erro ao quadrado, que pune muito os erros grandes, mas é sensível a outliers) e o Huber (MSE para erros pequenos, MAE para grandes).

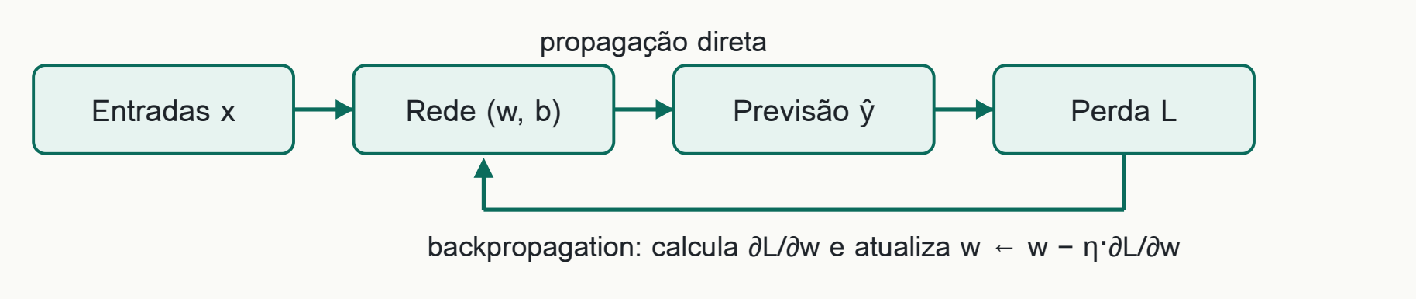
Como aqui o problema é de **classificação**, usamos a **entropia cruzada** (cross entropy), extensão da binary cross entropy para mais de duas classes: $L = -\Sigma\, y_{□} \cdot \log(\hat{y}_{□})$. Como o rótulo correto vale 1 e os demais 0, a perda se reduz a **−ln(probabilidade dada à classe certa)**.



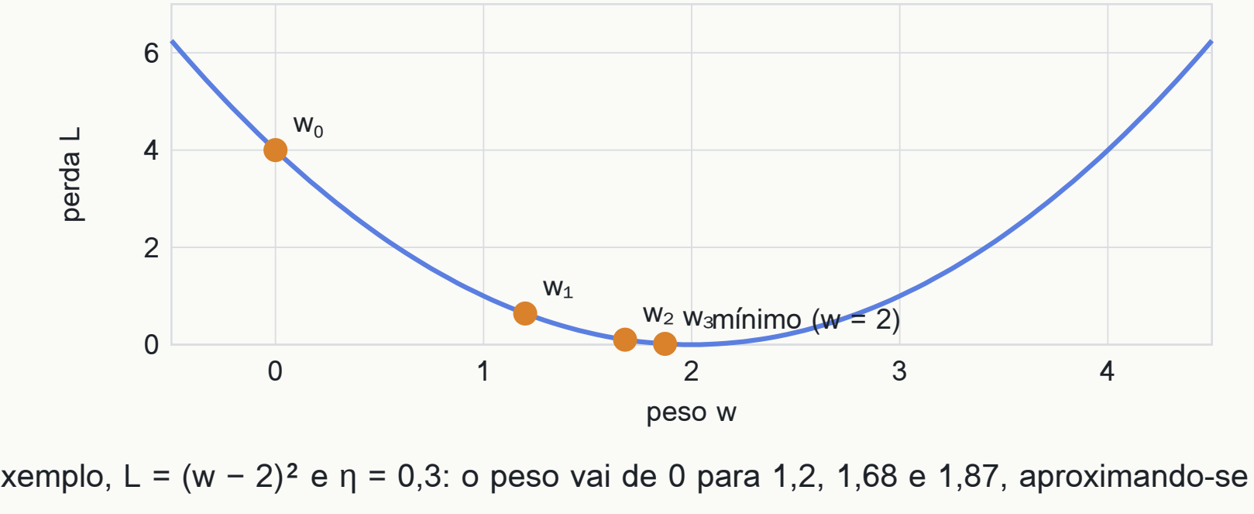
Se a rede dá 90% à classe correta, $L = -\ln 0{,}9 = 0{,}11$: quase nenhuma punição. Se dá 10%, $L = -\ln 0{,}1 = 2{,}30$: punição pesada. A curva dispara quando a probabilidade da classe certa tende a zero, então a rede é corrigida com força quando erra com confiança, o que o erro quadrático não faz.

Backpropagation

Backpropagation é o **treino da rede**: ela aprende com os erros. A perda L depende de todos os pesos, e o objetivo é achar os pesos que a tornam mínima. Em cada iteração:



1. **Propagação direta:** a rede calcula a previsão $\hat{y}$.
2. **Perda:** compara $\hat{y}$ com o rótulo e calcula L.
3. **Backpropagation:** pela regra da cadeia, calcula a inclinação $\partial L/\partial w$ (a derivada, ou reta tangente) de cada peso, da saída até a entrada.
4. **Atualização (descida do gradiente):** $w \leftarrow w - \eta \cdot \partial L/\partial w$, andando no sentido em que a perda diminui.



No exemplo, $L = (w - 2)^2$ e $\eta = 0{,}3$: o peso vai de 0 para 1,2, 1,68 e 1,87, aproximando-se do mínimo em w = 2. A **taxa de aprendizado** η é o tamanho do passo: pequena demais deixa o treino lento e ele pode acabar sem chegar ao mínimo; grande demais salta por cima do mínimo e a perda pode até subir. O **Adam** ajusta η para cada peso durante o treino, partindo de 0,001. O ciclo se repete por épocas e termina após um número fixo de épocas ou ao atingir uma tolerância (por exemplo, uma acurácia), valendo o que ocorrer primeiro.

> **O que é uma época?** É uma passada completa por todo o conjunto de treino. Os dados são divididos em lotes (mini-batches) e, a cada lote, ocorrem propagação direta, cálculo da perda, backpropagation e atualização dos pesos. A perda de treino cai a cada época; para evitar sobreajuste (overfitting), monitora-se a perda em um conjunto de validação e aplica-se parada antecipada (early stopping) quando ela deixa de melhorar.

Item	Escolhido	Alternativas	Motivo
Arquitetura	MLP 6→8→4→3	Redes maiores, LSTM	Dados tabulares, cabe no ESP32
Ativação oculta	ReLU	Sigmoide, tanh	Baixo custo, sem saturação
Ativação de saída	Softmax	Sigmoide	Classes mutuamente exclusivas
Função de perda	Entropia cruzada categórica	MSE	Classificação multiclasse
Otimizador	Adam (η = 0,001)	SGD	Convergência rápida
Execução	Treino no PC, inferência no ESP32	Treino embarcado	Limite de memória e processamento

7. Resultados esperados

- O ESP32 classificar cada leitura no estado correto, em tempo real.
- Metas de desempenho: acurácia ≥ 95% no conjunto de teste e latência de inferência inferior a 1 ms.
- Os estados raros (SOBRECARGA e CRÍTICO), se não tiverem exemplos suficientes, serão tratados por um alarme de limite fixo no programa.

8. Limitações

- Os dados cobrem poucos dias de operação; mais tempo de coleta deixa o modelo mais confiável.
- Como o rótulo vem de faixas de corrente, o modelo reproduz uma regra; seu ganho está em decidir com mais estabilidade, usando também potência e tendência.

9. Glossário

TinyML
Aprendizado de máquina que roda em equipamentos pequenos e baratos.

Rótulo
A resposta certa de cada exemplo (o estado da máquina).

Variável de entrada
Cada número que o modelo recebe para decidir (corrente, potência...).

Peso
Número ajustado no treino que define como o modelo decide.

Inferência
O modelo já treinado dando a resposta para uma leitura nova.

Função de ativação
Função não linear aplicada à saída de cada neurônio (aqui, ReLU e softmax).

Função de perda
Medida do erro entre previsão e rótulo, que o treino minimiza (aqui, entropia cruzada).

Backpropagation
Algoritmo que calcula o gradiente do erro em cada peso, da saída para a entrada.

Época
Uma passada completa da rede por todo o conjunto de treino.

Normalização z-score
Conversão de cada variável para média 0 e desvio-padrão 1, para que todas tenham o mesmo peso no treino.

CSV
Arquivo de texto simples com os dados em linhas e colunas.

Fontes

WEG, Folha de Dados do motor W22 IR3 Premium 3 cv 4P, código 11397215, p. 1/5. Schmersal, Manual do inversor Praxi 20 (2023), sinais de monitoramento p. 69 e registradores Modbus p. 158 (numeração impressa).